What and When

Storage is a place where we store some data. The simplest storage is JavaScript memory, where we can store variable values.



When do we need storages?

In any application, we need to store some data.

Data storage is not a necessity for simple static sites, which sometimes don't even need JavaScript.

What the problem?

Some storages are overused while others are in the shadows.

Local storage is often used as the default storage. While storages such as **Session Storage** and **Indexed DB** are relatively less popular. This video will help to understand which tool is best suited for a particular case.



What storages we will discuss?

- URL
- Cookies
- Session Storage
- Local Storage
- Cache API
- Indexed DB
- Remote database

What we leave out of the scope?

- File System API (Poor browsers support)
- Web SQL (Deprecated)

URL

Considering a URL as a storage is not obvious. The URL has ***search parameters*** OR a ***query string***.
We usually think of it as query parameters. However, we can also use it as a storage.

URL search parameters will survive a page refresh, but they will be lost when the page is closed.
Therefore, we cannot consider URL search parameters as a safe place to store data.

When Search Parameters may be useful?

It has one unique feature. It is easy to share it with other users and save it in your browser bookmarks. Other storages don't have such features. For example, we can't easily save the page state in **Local Storage** and share it with other users in the future.

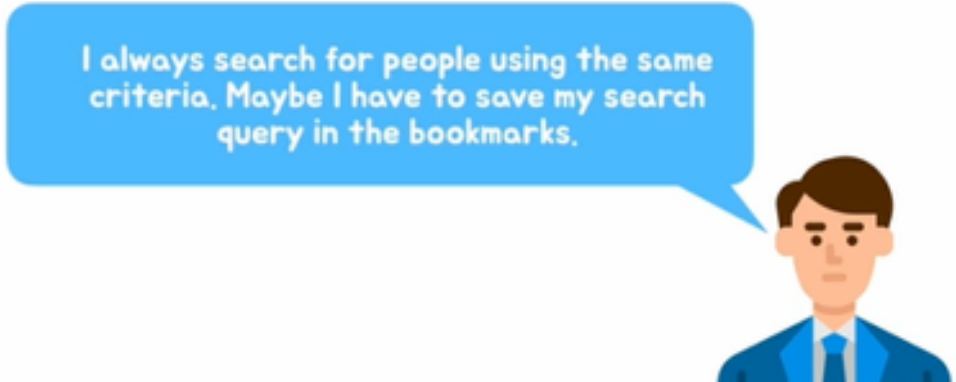
Pros and Cons

Pros

- **Easy to use.**
- **Sharing.** Allows to share data easily.

Cons

- **Fragile.** It can be easily removed or lost.
- **Not secure.** All data is visible.
- **Small Capacity.** Suitable mainly for short and primitive (string, number or boolean value) data.



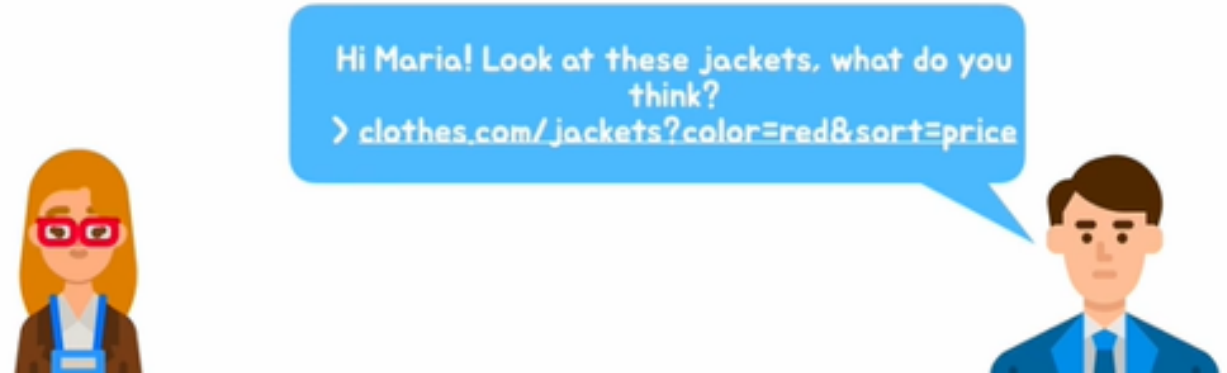
Use cases

Current page's state

Ideal for applications where you want to share state, such as a configured view on a dashboard or map.

Navigation state

Storing page numbers, filter settings or search queries during a user's navigation session.



URL



Cookies

Cookies are another storage with unique features.

- Cookies may be set by the server.
- Cookies will be automatically sent to the server.

Pros

- Widely supported and easy to use
- Increased protection against XSS attacks

Cons

- Limited storage capacity (about **4KB** per origin).
- Sent with every HTTP request, increasing **HTTP overhead**.

Use cases

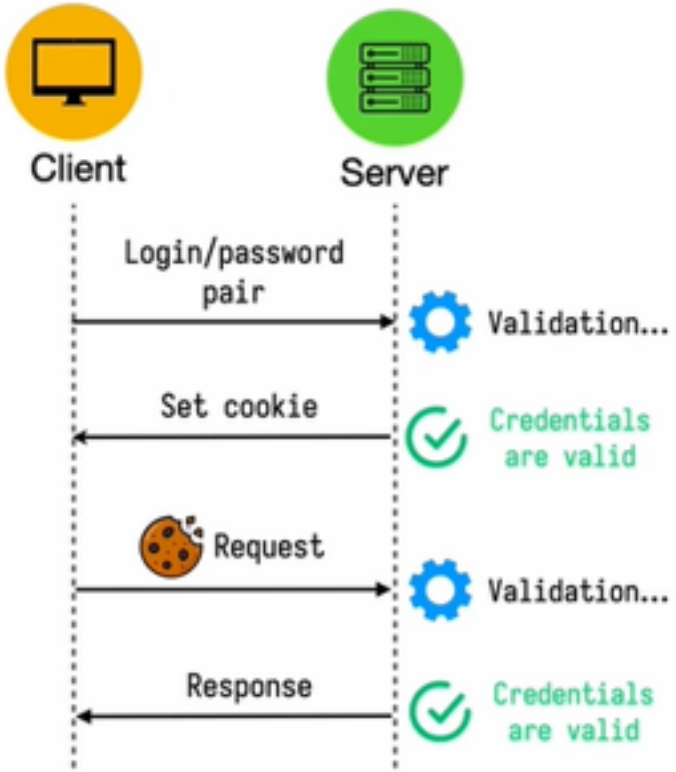
- Session Management
- Personalization
- Tracking users behaviour

1 Session Management

Cookies are:

- Can be set by the server
- Are automatically sent with each request.

These two features make Cookies an ideal tool for managing **user sessions**. This process does not require JavaScript to manage the sending of session information.

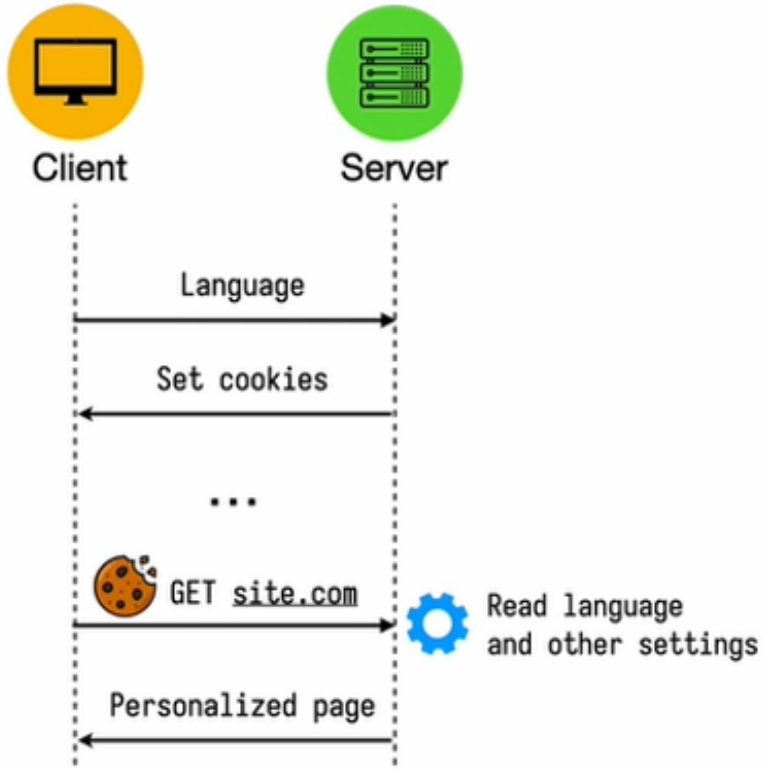


2 Personalization

Since cookies are sent with each request, we can display a personalized page on the server when the browser requests the page for the first time.

Let's take a look at how this works:

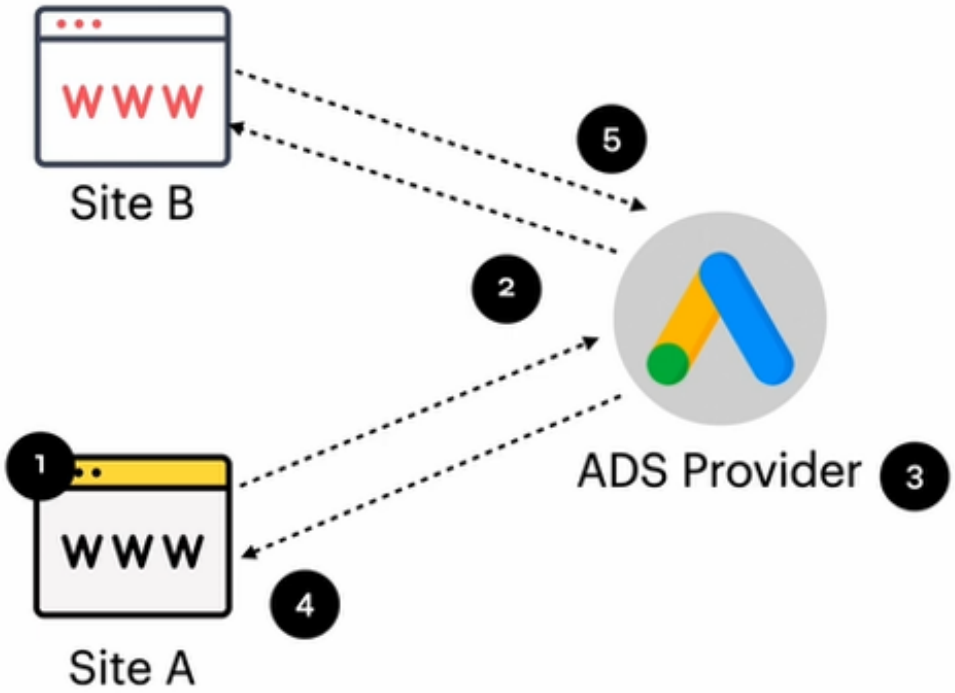
- **First visit.** User visits the site > Selects language and other settings > Browser sends the settings to the server > Server sets cookies for the browser.
- **Next visit.** Browser sends a request to the page, cookies are sent automatically > Server reads cookies and returns a personalized page.



3 Tracking users behaviour

How cookies may help to track user's behaviour?
Let's look on example.

1. **Site A** places advertisements provided by an advertising provider. A user visits this site.
2. The **ADS script** sends user information to the **ADS Provider Server.**
3. The **ADS provider Server** generates a unique `UserID` based on the information sent.
4. The **ADS provider Server** sets cookies with the `UserId` to the user's browser.
5. Later, the user visits another site that connects to the same **ADS Provider**. When the **ADS script** requests an advertisement, it also sends cookies with the `UserID` to the **ADS Provider Server**.



Local Storage

Pros

- **Persistent storage**
- **Easy to use**

Cons

- **Limited capacity.** Typically browsers provide 5MB for each origin, but in some cases (web-kit based browsers) this may only be 2.5MB because JavaScript stores strings in UTF-16 format.
- **Slow synchronous API**. LocalStorage does not perform operations asynchronously. This means that LocalStorage operations will take up the main thread, which can cause the application to slow down.
- **Not available inside Web or Service Workers.** Because of the synchronous API (which must be executed inside the main thread) it is not available there.
- **Only strings are supported.** Although LocalStorage is easy to use, it provides a fairly primitive set of features. It does not support any data types other than strings. This means that a developer will have to encode/decode the data every time they need to read/write it. Such processes require additional resources and may slow down the application.

Use cases

Persistent Client Storage

We will discuss later that Local Storage is not completely persistent, and data can still get lost (which is why Local Storage should not be used for important data), but in some cases the suggested persistence is sufficient. We can locally store various user settings and application data such as the selected theme, language, and other settings. This is useful when we can't or don't want to use a database.

Offline usage

In the case where we need offline mode, Local Storage is the easiest solution to store data. For example, when a user is offline, he performs some operations. We can save these operations in Local Storage and send them to the server later.

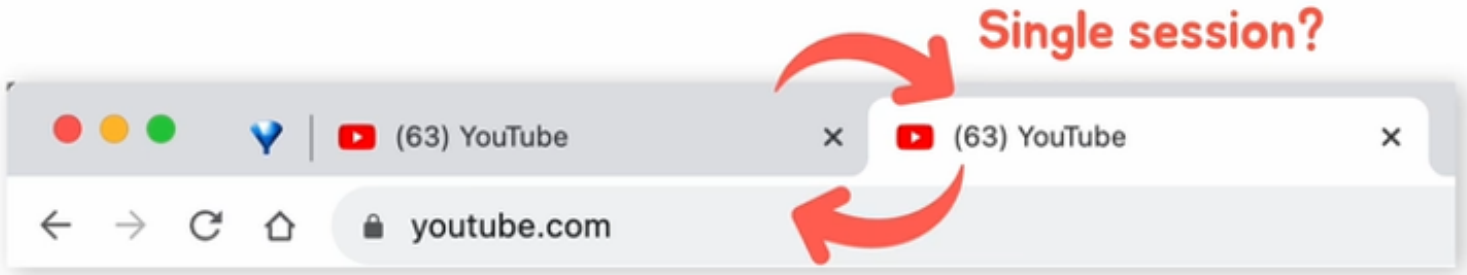
Saving Drafts

Draft is a common case for applications that involve lengthy user input, such as blogging platforms (Medium), where users can edit a single text or form for an extended period of time.

Session Storage

Session storage stores data ***within a single session***. But what does it mean?

- When a user opens a web page in a browser, a session is created.
- During tab refresh and restore (when the browser has been closed) Session Storage saves the data.
- When the user explicitly closes the tab or the tab has been closed by the browser and not restored, the session is considered ***finished***. The Session Storage for such a tab will be ***cleared***.



Two more points:

- Session Storage does not transfer data between tabs, even if they have the same origin. *One tab is one session.*
- The ***exception*** to the previous rule is that if a tab has been ***duplicated*** (not just a URL copied and pasted), the original tab and the duplicated tab will share the same session store.

Pros and Cons

Pros

- Similar to **LocalStorage**, but is cleared when the session is terminated (closing a browser tab). We should not care about data eviction.
- In some browsers (e.g. IE) Session Storage has a larger size (up to 12 MB) than **Local Storage**. Session Storage also has a separated **Quota**.

Cons

- Inherited from **Local Storage**

Use cases

Caching

When data was requested from the server once, we can put it in the storage and use it during the current session. The Session Storage provides an automatic cleanup mechanism, so we don't have to worry about cache invalidation.

User/System data shared across screens

We can use Session Storage to share data between screens, instead of creating a global object that won't survive a page reload. Example: A wizard widget with multiple steps.

Global configuration

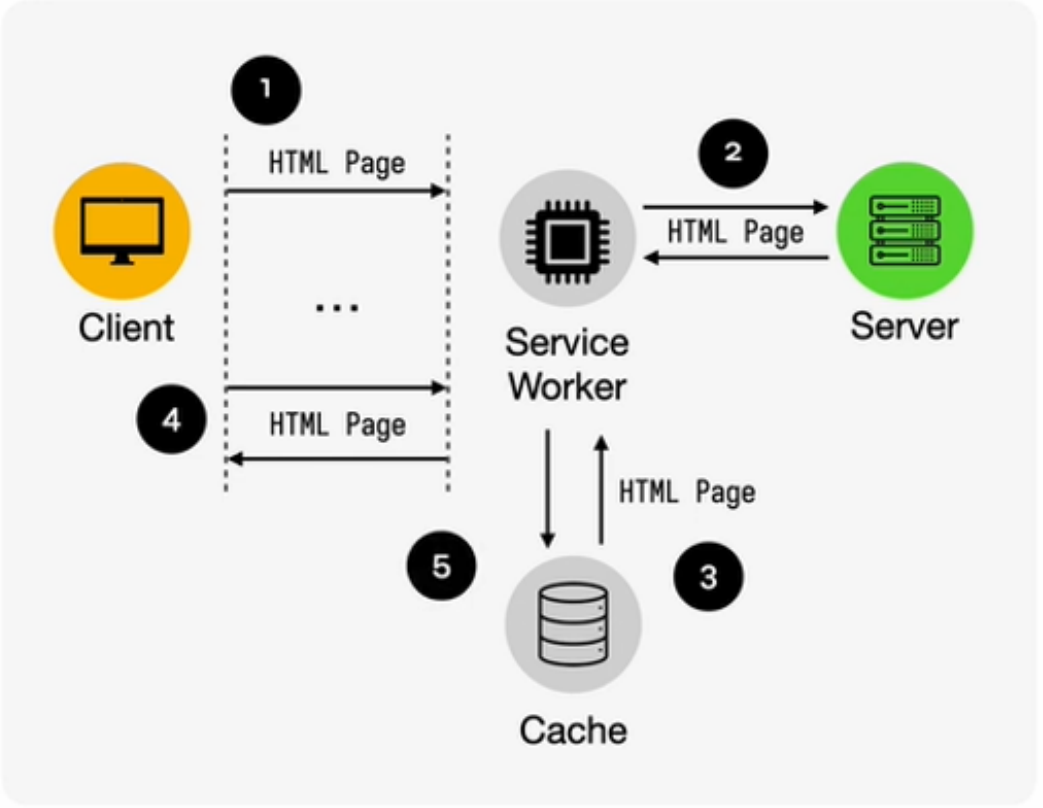
When we need to store some global configuration of the application, we can use Session Storage instead of global variables or storage managers.

Cache API

Cache API is separate from other types of storage. This storage is only for storing **request/response pairs**. The Cache API is available inside **Web Workers** and **Service Workers**.

It typically works as follows:

1. A client requests an HTML page.
2. Service Worker intercepts the request. The request was not found in the cache, so the Service Worker requests the page from the server.
3. Upon receiving the HTML page, Service Worker stores it in the cache.
4. Lately, the client requests the same HTML page again.
5. Request for the page is intercepted by Service Worker. Request is found in the cache, therefore, it would be returned to the client without request to the Server.



Use cases

Offline mode

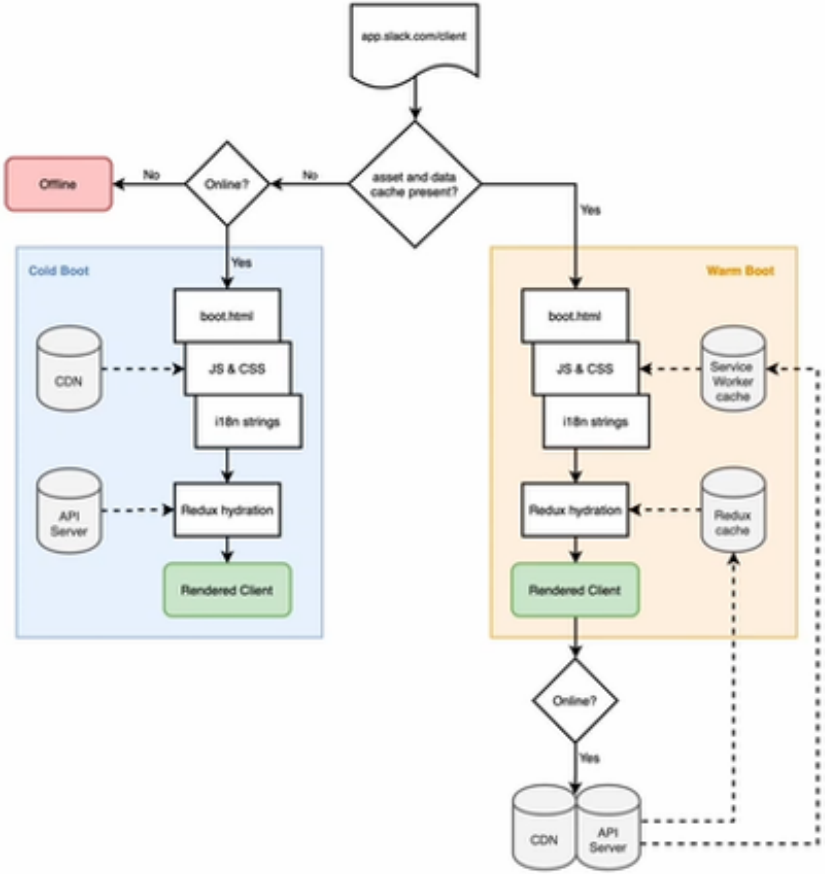
Cache API (and Service Workers) is very useful in cases when we need to provide Offline Mode. If assets (HTML, CCS and other static files) were saved in cache, we can use application without a network connection.

Reduce the number of network requests by reusing assets

Even, if we don't need to implement Offline mode, Cache API may be useful for resource saving and increasing loading speed. When asset was requested once, it may be saved to the Cache and reused in the future.

Real Use Case. Slack.

Service Workers and Cache are used in **Slack** for efficient app loading and offline functionality. By caching assets like HTML, JavaScript, CSS, and storing data in IndexedDB, Service Workers enable quick app startups (***"warm boots"***) and usability even without an internet connection, enhancing user experience significantly.



Cache API

Browser's Cache

Why do we need an additional Cache API when we already have a cache in the browser?

Browser's Cache

- Controlled by the browser based on the `Cache-Control` HTTP header.
- Not flexible.
- Mainly used to economy user's network traffic.

Cache API

- Completely controlled by developers. Does not depend on the `Cache-Control` HTTP Header.
- Very flexible.
- Mainly used for Offline mode.

How browser's cache works?

When the browser receives a response and it can be cached, there are two possible scenarios:

- **Cache Control is enabled.** In this case, the file will be stored on disk (in the browser cache). The next time the file will be retrieved from disk without a request to the server. This allows such files to be used offline and saves resources when the user is online. However, `Cache-Control` settings should be carefully adjusted to avoid returning outdated content to users.

- **Cache-Control is disabled.** In this case, the browser has to make a request to the server. Currently, the usual scenario is to send a request to the ***CDN***. If the file is relevant, the CDN will return a `304` status code and an empty body. This means that the browser can retrieve the resource from the local cache.

Indexed DB

Pros

- **Large capacity.** Allows you to store large amounts of structured data.
- **Powerful.** Supports advanced features such as indexing, transactions, and versioning.
- **Fast.** Supports asynchronous operations.
- **Accessibility.** Available in Web and Service Workers.

Cons

- **Steeper learning curve.** More difficult to develop and maintain.



It is not recommended to use IndexedDB as is. Wrappers like idb can hide the complexity of IndexedDB and make it much more convenient to use.

Possible issues

Intelligent Tracking Prevention (ITP)

Safari's data (including **Local Storage** and **IndexedDB**) is deleted after 7 days of inactivity. This means that we cannot consider the browser storage as a long-term storage of user data; it can only use it as a one-time cache.

Doesn't work in Private mode in Firefox

Also, we don't have an API to detect when a site is open in private mode and notify users of possible problems when using private mode. The only way is to try to open a connection with IDB and show a possible solution in case of failure.

Quotas are confusing

Quotas vary from browser to browser. For example, in Firefox, the quota is shared among all subdomains. We can't create a predictable way to handle quota exceed errors that covers all browsers.

Use cases

Storing large amounts of data on the client side.

With its high capacity and fast asynchronous API, IndexedDB is a good choice for storing large amounts of data.

Storing data with complex structure.

IndexedDB supports almost all JavaScript types, except for some, such as functions, for example. If we need to store such data, IndexedDB is a good choice.

Storing binary files

IndexedDB supports binary file storage, which gives us many possibilities such as: storing media content for offline use or uploading media files to IndexedDB.

Offline-first applications

Offline-first application must work as well offline as it does online. The other application simply has no backend and relies only on client-side storage.

Quotas

Some storages share same ***quota***.
Quota is the disk space that the browser allocates for storing data.

Following storages share same quota:

- Local storage
- Indexed DB
- Cache (Cache API)
- Web SQL (Deprecated)
- FileSystem API (poor browsers support)

These storages have individual quotas:

- Cookies
- Session storage
- Browser's cache

Shared quota includes:

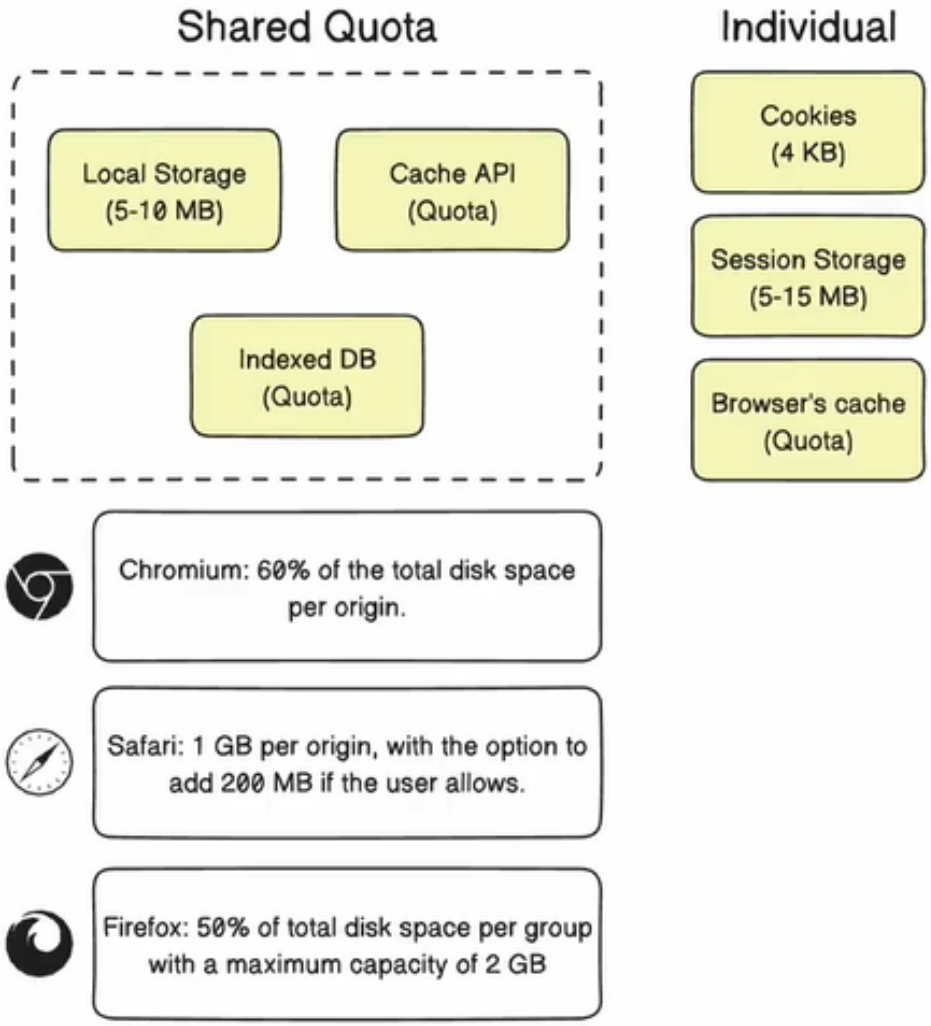
- **Local storage.** Maximum capacity is 5-10 MB or the size of the shared quota.
- **Cache API.** Maximum capacity is equal to the size of the shared quota.
- **IDB.** Maximum capacity is equal to the size of the shared quota.

Shared quota size:

- **Chromium-based browsers.** 60% of the total disk space for each origin.
- **Safari.** 20% of the total disk space per origin.
- **Firefox.** 10% of total disk space per group (domain + subdomains) with a maximum of 10 GB in ***best-effort mode.***

Individual quotas:

- **Cookies.** 4 KB.
- **Session storage.** 5-15 MB.
- **Browser cache.** Controlled by the browser.



Quotas

Eviction policy

Browsers use one of two policies for eviction:

- Best-effort (default)
- Persistent

Best-effort means that if the quota is exceeded, the browser will automatically remove data from the storage according to the Least Recently Used (LRU) policy.
Persistent mode means that the browser will not automatically delete data from the storage. When the quota is exceeded, the write operation throws a `QuotaExceededError` error. When using Persistent mode, it is important to wrap the write operation in a `try/catch` block.

Persistent mode is needed when our application is highly dependent on offline storage, for example, if it is **offline-first.**



In Firefox, to switch a web page to persistent mode, we have to ask for user permissions. In Safari and Chromium based browsers allow or disallow switching to persistent mode happens automatically based on user behaviour.

Database

Pros

- **Reliability.** The database provides the highest reliability compared to the other storage we discussed earlier.
- **Security.** It is much safer to store data on servers.
- **Data Sharing.** Client-side storage does not allow data sharing between user devices. Database helps to solve this problem.

Cons

- **Complexity.** Adding a database requires additional team resources or external resources.
- **High cost.** Maintaining the database requires additional resources.

Use cases

- **Sensitive data.** Client-side storages is vulnerable to XSS and other types of attacks.
- **Sharing.** Data that needs to be shared between devices.
- **Important data.** The database is the most reliable storage among others.

When we develop almost any Web application, there is a good chance that we will have a database on the server side. However, that doesn't mean that we have to use a database for all data.

Why not just store all the data in the database?

High Cost. We have to handle tons of user requests and maintain data on servers.

Slow interface. Since we have to retrieve data from the server, there may be a delay depending on the quality of the user's network connection and other factors.

Impossible to use offline. When the user does not have access to the internet, the application becomes useless.

What about Store Managers?

Storage ≠ Store Manager

Storage

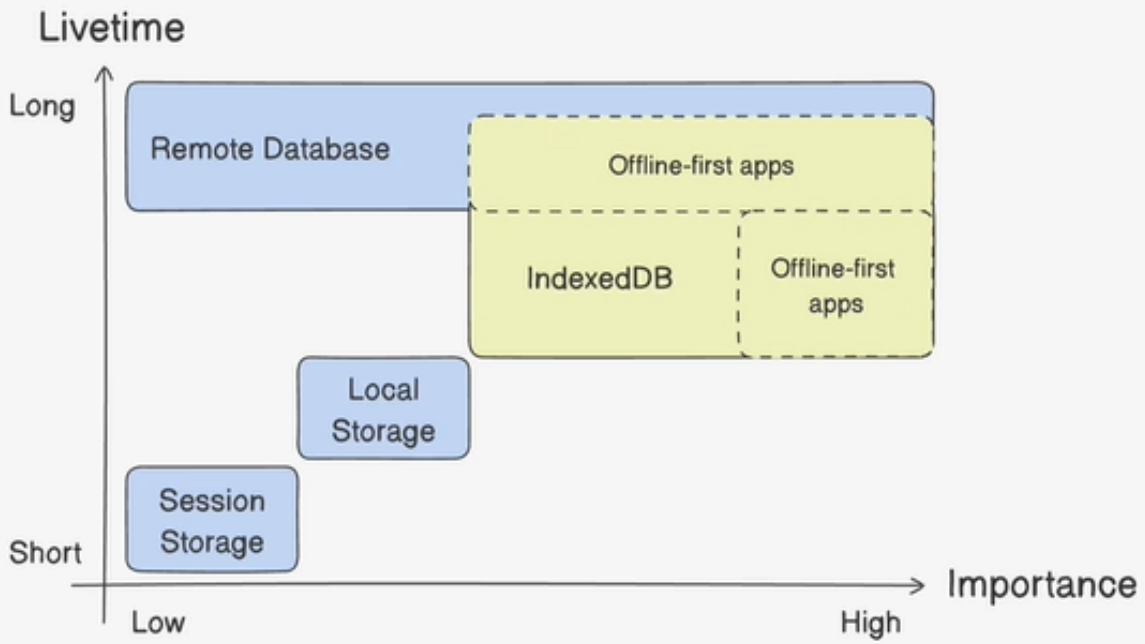
- Takes up physical space on the disk
- Provides an API for interactions
- Has varying capacities and lifetimes. Persistent storage stores data forever.

Storage Manager

- Does not provide physical disk space. Store Manager works on top of existing storage. Therefore, Store Manager can provide temporary or permanent storage. For example, Redux provides both options.
- Provides an interface for working with storage and facilitates data operations such as CRUD operations, subscription for updates, and others.



How to make a decision?



We left out **URL parameters**, **Cookies** and **Cache** because such storages are used in very specific cases.

We can choose the appropriate storage according to the lifetime and importance of the data.

For **Online-first applications** that are used only or mostly offline, we can use IDB even for long-lived and important data.

Real Example

Search

Users can search other users by name, city, age and other parameters. We can also sort search results by date, popularity, and more.

What type of storage can we use for this kind of data?

- Search parameters are not very important data. Loss of such data is not a big problem.
- Search parameters are short-lived data.
- It is quite possible that we may want to save the configuration of such filters in our browser bookmarks or share with friends.

Therefore, **URL parameters** are a good choice.



Real Example

Adding posts

When a user creates a new post, it is saved in the database.

Creating a post may take some time. Users may want to create a ***draft*** and come back to it later. We also need to decide where to store such drafts. We have 3 possible locations:

- **Local storage.** The simplest solution. Suitable when we don't need to store many drafts and when the draft is plain text.
- **Indexed DB.** More complex to maintain, but gives us more scalability. Suitable for many drafts, perhaps with a more complex structure (title, content, tags, attachments, etc.).
- **Database.** The most robust solution. Also allows data sharing between devices. Covers all possible cases, but has disadvantages: difficult to maintain and requires requests to the server to read/write data.

The final choice is based on the ***functional requirements*** of the application.

Why not Local Storage?

Local storage is good for simple data.

In our case:

- We are dealing with a complex structure.
- Each post, in addition to post's body, may contain various meta fields.
- Posts can be long, and since Local Storage API is ***synchronous***, reading/writing such amounts of data can slow down the application.
- Local Storage does not support ***binary files***, which can be useful offline, if users want to add media files to the post.

Why not Indexed DB?

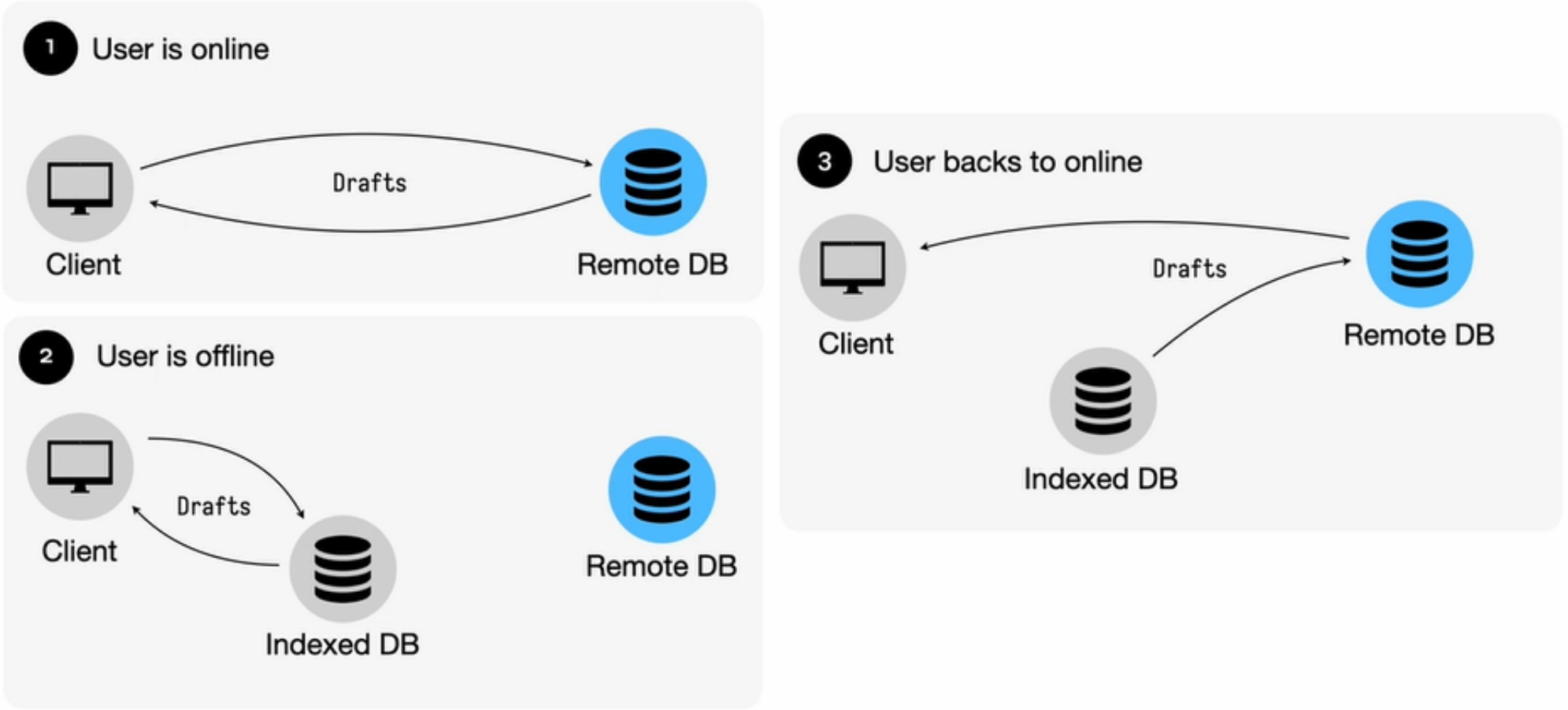
Indexed DB is good for storing drafts when the user is ***offline***.
However, it should not be used in usual cases.

Indexed DB:

- Does not allow data sharing between devices.
- Data storage is not completely reliable. Data loss is still very likely.

We will use a Remote Database as a persistent solution.

How it works together



Real Example

Summary

Cookies

Use cases: Sessions management, tracking, personalization
Key advantages: secure, send to the server automatically.
Key disadvantages: Small capacity, increases HTTP overhead

URL Parameters

Use cases: page's state
Key advantages: sharing
Key disadvantages: small capacity, increases URL length

Local Storage

Use cases: storing long-term data
Key advantages: simplicity
Key disadvantages: slow API, small capacity, limited features

Session Storage

Use cases: global data
Key advantages: automatic eviction policy
Key disadvantages: inherited from Local Storage

Indexed DB

Use cases: storing long-term data, offline usage
Key advantages: capacity, powerful, fast asynchronous API
Key disadvantages: steeper learning curve,
require wrappers to hide the complexity.

Remote DB

Use cases: storing long-term data
Key advantages: secure, reliable
Key disadvantages: expensive